UBIRA-V

Especificacao de Projeto de um Coprocessador Vetorial RISC-V

Lucas Farias Martins

12 de outubro de 2026

Resumo

Este documento registra o estado atual do projeto de um coprocessador vetorial inspirado na arquitetura RISC-V. O foco desta etapa é a especificação funcional executável em Python, anterior à migração para SystemC e à implementação RTL em SystemVerilog.

A configuração física adotada é VLEN igual a 128 bits. O modelo está sendo generalizado para suportar elementos inteiros de 8, 16 e 32 bits, bem como grupos de registradores com LMUL igual a 1, 2, 4 ou 8. A especificação também descreve operações inteiras, cargas e stores contíguos, execução sequencial de programas, política de cauda e processamento de vetores maiores que VLMAX por strip-mining.

O documento distingue decisões de projeto, comportamento pretendido, funcionalidades presentes no modelo e pendências identificadas durante a execução dos testes.

Sumário

1 Objetivo e escopo **3**

2 Estado do projeto **4**
2.1 Decisões estabelecidas 4
2.2 Estado observado nos testes 4

3 Parâmetros arquiteturais **4**
3.1 Largura física 4
3.2 Largura do elemento 4
3.3 LMUL 5
3.4 VLMAX 5

4 Estado arquitetural do modelo **5**
4.1 Representação física dos registradores 5
4.2 Grupos de registradores 6

5 Especificação executável **6**
5.1 Programa 6
5.2 Execução sequencial 6
5.3 Trace 6

6 Strip-mining e tamanho residual **7**

7 Semântica funcional **7**
7.1 Operações inteiras elemento a elemento 7
7.2 Política de cauda 8
7.3 Política de máscara 8
7.4 Loads e stores contíguos 8

8 Modelo em software **9**

9 Verificação existente **9**

10 Pendências e riscos conhecidos **10**

11 Critérios para o próximo marco **10**

12 Migração futura para SystemC **11**

13 Histórico deste documento **11**

1 Objetivo e escopo

O objetivo do projeto é desenvolver um coprocessador vetorial para pesquisa arquitetural, com foco futuro em uma implementação sintetizável para ASIC. O fluxo de desenvolvimento definido é descrito abaixo:

1. Modelo funcional executável em Python;
2. Modelo temporal ou de arquitetura em SystemC;
3. Implementação RTL em SystemVerilog;
4. Simulação e verificação com Xcelium e SimVision;
5. Síntese e análise de área e temporização com Genus.

Esta versão documenta o modelo funcional. Ela não especifica pipeline, latência de hardware, protocolo de memória RTL, exceções arquiteturais completas, interface com um processador escalar ou desempenho físico.

2 Estado do projeto

2.1 Decisões estabelecidas

Parâmetro	Valor ou decisão
VLEN	128 bits
ELEN	32 bits
Elementos inteiros modelados	SEW de 8, 16 e 32 bits
LMUL inteiro modelado	1, 2, 4 e 8
Registradores vetoriais físicos	32
Endianness da memória	Little-endian
Acesso vetorial à memória	Unit-stride, contíguo
Política de cauda	Tail-undisturbed
Política de máscara	Mask-undisturbed

Tabela 1: Parâmetros e decisões atuais do projeto.

2.2 Estado observado nos testes

Na última execução registrada de `pytest -v`, foram coletados 38 testes: 31 passaram e 7 falharam. As falhas apontam para a mesma causa imediata: o nome `SEW` não está definido em `model/executor.py`, embora seja usado pelo handler de `vsetvli`.

Portanto, os testes de estado, operações básicas, SEW/LMUL e geração estática do programa de strip-mining passaram. Os testes que executam programas pelo `Executor`, inclusive os testes end-to-end de strip-mining, falharam ao processar a primeira instrução `vsetvli`.

Este resultado não deve ser descrito como uma suíte totalmente aprovada. A correção do import e a repetição da suíte são pendências antes de considerar a especificação executável validada integralmente.

3 Parâmetros arquiteturais

3.1 Largura física

Cada registrador vetorial físico tem largura:

$$VLEN = 128 \text{ bits} = 16 \text{ bytes}.$$

O banco contém 32 registradores físicos, indexados de 0 a 31.

3.2 Largura do elemento

A largura do elemento selecionado, denominada SEW, pode assumir os valores suportados pelo modelo:

$$SEW \in \{8, 16, 32\} \text{ bits}.$$

O maior elemento suportado é:

$$ELEN = 32 \text{ bits}.$$

Nesta etapa, os valores são tratados como inteiros sem sinal para representação e como inteiros de largura fixa para as operações aritméticas. A aritmética modular é definida pela largura do elemento.

3.3 LMUL

A configuração LMUL suportada nesta versão é:

$$LMUL \in \{1, 2, 4, 8\}.$$

A implementação não inclui multiplicadores fracionários de registradores, como LMUL igual a |1/2|, |1/4| ou |1/8|.

3.4 VLMAX

Para os valores inteiros de LMUL implementados, o número máximo de elementos ativos em um grupo vetorial é:

$$VLMAX = \frac{VLEN}{SEW} \times LMUL$$

Exemplos:

SEW	LMUL	VLMAX
8	1	16
16	1	8
32	1	4
32	2	8
8	4	64

O comprimento efetivo do vetor, `vl`, é limitado a VLMAX:

$$vl = \min(vl_{\text{requested}}, VLMAX).$$

O tratamento de valores negativos para `vl` ainda deve ser formalizado e validado explicitamente.

4 Estado arquitetural do modelo

O estado funcional do coprocessador contém:

- 32 registradores vetoriais físicos;
- o valor corrente de `vl`;
- o valor corrente de `vtype`, composto por SEW e LMUL;
- a memória usada pelos testes de load e store.

4.1 Representação física dos registradores

Cada registrador vetorial físico é representado como uma sequência de 16 bytes. Essa representação permite reinterpretar o mesmo estado físico sob diferentes valores de SEW.

Por exemplo, os mesmos 16 bytes correspondem a:

- 16 elementos de 8 bits;
- 8 elementos de 16 bits;
- 4 elementos de 32 bits.

A ordem dos bytes dentro de um elemento é little-endian.

4.2 Grupos de registradores

Quando LMUL é maior que 1, um registrador lógico ocupa um grupo consecutivo de registradores físicos. O tamanho do grupo é igual a LMUL.

O índice inicial do grupo deve ser múltiplo de LMUL. O grupo também deve permanecer dentro dos 32 registradores físicos.

Exemplos:

- LMUL igual a 2: índices-base válidos são 0, 2, 4, . . . , 30;
- LMUL igual a 4: índices-base válidos são 0, 4, 8, . . . , 28;
- LMUL igual a 8: índices-base válidos são 0, 8, 16 e 24.

A leitura e escrita de elementos lógicos concatenam, em ordem crescente, o conteúdo dos registradores físicos que compõem o grupo.

5 Especificação executável

5.1 Programa

O programa funcional é representado como uma lista ordenada de pares compostos por mnemônico e operandos nomeados.

Exemplo conceitual:

```
[
    ("vsetvli", {"vl": 4, "sew": 32, "lmul": 1}),
    ("vadd.vv", {"vd": 3, "vs1": 1, "vs2": 2})
]
```

A classe `Instruction` valida o mnemônico e os operandos de cada instrução antes da execução. O executor processa a lista em ordem, aplicando cada instrução ao estado do modelo.

5.2 Execução sequencial

O executor atual é funcional e sequencial:

1. valida ou normaliza a instrução;
2. identifica o handler correspondente;
3. executa a operação sobre o modelo;
4. registra os efeitos observáveis no trace.

Não há modelagem de ciclo, pipeline, issue múltiplo, hazards, interrupções, flush ou concorrência de memória.

5.3 Trace

O trace registra uma entrada por instrução executada. Os campos atuais incluem:

- índice da instrução;
- mnemônico;
- operandos;
- valor de `vl` durante a execução;
- registradores de destino observados;

- escritas de memória observadas, quando aplicável.

O trace pode ser serializado em JSON. Valores incluídos no JSON devem ser tipos serializáveis, como inteiros, strings, listas e dicionários. Valores de memória não devem ser armazenados como objetos `bytes` sem uma conversão explícita.

6 Strip-mining e tamanho residual

O suporte atual a strip-mining gera uma sequência de instruções para processar um vetor lógico maior que VLMAX.

Para um número total de elementos $|N|$, cada iteração escolhe:

$$vl_{\text{iter}} = \min(N_{\text{remaining}}, VLMAX).$$

Após cada iteração:

$$offset_{\text{elements}} \leftarrow offset_{\text{elements}} + vl_{\text{iter}},$$

$$N_{\text{remaining}} \leftarrow N_{\text{remaining}} - vl_{\text{iter}}.$$

Um kernel de soma contígua executa, em cada iteração:

1. configuração de `vl`;
2. load do trecho corrente do vetor A;
3. load do trecho corrente do vetor B;
4. soma dos registradores vetoriais;
5. store do trecho corrente do resultado.

Para VLEN igual a 128, SEW igual a 32 e LMUL igual a 1, $|VLMAX = 4|$. Um vetor de 10 elementos é processado com comprimentos:

$$4,\ 4,\ 2.$$

O avanço de endereço entre iterações é calculado em bytes:

$$offset_{\text{bytes}} = offset_{\text{elements}} \times element_bytes.$$

A implementação inicial de strip-mining foi escrita para SEW igual a 32 e LMUL igual a 1. Sua generalização para SEW e LMUL variáveis ainda é uma pendência.

7 Semântica funcional

7.1 Operações inteiras elemento a elemento

O conjunto de operações aritméticas e lógicas considerado nesta etapa é:

- soma vetorial;
- subtração vetorial;

- AND bit a bit;
- OR bit a bit;
- XOR bit a bit.

Para uma operação binária com destino $|vd|$ e fontes $|vs1|$ e $|vs2|$, cada elemento ativo $|i|$, em que $|0 \leq i < vl|$, é calculado como:

$$vd[i] = f(vs1[i], vs2[i]).$$

O resultado é truncado para SEW bits. Para soma e subtração, isso corresponde a aritmética modular:

$$vd[i] = f(vs1[i], vs2[i]) \bmod 2^{SEW}.$$

7.2 Política de cauda

A política definida para esta versão é tail-undisturbed. Elementos do destino com índice $|i \geq vl|$ preservam seu valor anterior.

$$vd_{\text{new}}[i] = vd_{\text{old}}[i], \qquad i \geq vl.$$

7.3 Política de máscara

A interface funcional aceita, quando aplicável, uma máscara representada por uma sequência de valores booleanos. Para um elemento ativo dentro de `vl`:

$$vd_{\text{new}}[i] = \begin{cases} f(vs1[i], vs2[i]), & mask[i] = 1, \\ vd_{\text{old}}[i], & mask[i] = 0. \end{cases}$$

A política é mask-undisturbed. A validação de tamanho da máscara deve garantir que todos os índices consultados existam. O formato arquitetural final do registrador de máscara ainda não foi especificado.

7.4 Loads e stores contíguos

O modelo inclui cargas e stores unit-stride. Para um elemento de largura SEW:

$$element_bytes = \frac{SEW}{8}.$$

O endereço do elemento $|i|$ é:

$$address(i) = base + i \times element_bytes.$$

A memória é endereçada por byte e usa little-endian. Loads respeitam `vl`, máscara e política de cauda. Stores escrevem apenas os elementos ativos, isto é, aqueles com $|0 \leq i < vl|$ e máscara ativa.

O nome atual das operações de memória precisa ser consolidado. O modelo foi generalizado para operações dependentes do SEW, mas o executor e alguns testes ainda usam nomes `vle32.v` e `vse32.v`. Esses nomes sugerem EEW fixo de 32 bits e podem não corresponder à largura efetivamente usada pelo modelo.

8 Modelo em software

A organização lógica atual do modelo Python é:

```
model/
    vector_state.py
    vector_isa.py
    memory.py
    golden_model.py
    instruction.py
    executor.py
    trace.py
    strip_mining.py
    inspect_trace.py
    tests/
```

Responsabilidades principais:

`vector_state.py` Estado arquitetural, configuração SEW/LMUL e acesso lógico aos registradores vetoriais.

`vector_isa.py` Semântica funcional das operações vetoriais.

`memory.py` Memória endereçada por byte usada pelo modelo.

`golden_model.py` API de alto nível para o estado, operações e memória.

`instruction.py` Representação e validação de instruções do programa.

`executor.py` Interpretação sequencial do programa e geração do trace.

`trace.py` Registro e serialização dos efeitos de execução.

`strip_mining.py` Geração de programas de processamento por blocos com residual.

`inspect_trace.py` Execução de um caso demonstrativo e impressão do trace.

9 Verificação existente

A suíte de testes cobre, entre outros casos:

- aritmética vetorial e overflow modular;
- política tail-undisturbed;
- operações mascaradas;
- loads e stores contíguos;
- saturação de `vl` em VLMAX;
- VLMAX para diferentes combinações de SEW e LMUL;

- grupos de registradores com LMUL maior que 1;
- reinterpretação dos mesmos bytes com SEW diferente;
- construção de programas strip-mined;
- endereços crescentes entre iterações;
- execução end-to-end de vetores maiores que VLMAX;
- serialização e repetibilidade do trace.

O último resultado observado foi:

31 testes aprovados e 7 testes falhos, de um total de 38.

As sete falhas ocorreram no handler de `vsetvli` do executor, porque `SEW` foi referenciado sem estar importado em `model/executor.py`. As falhas afetam a execução de programas, incluindo os testes end-to-end de strip-mining.

10 Pendências e riscos conhecidos

1. Importar `SEW` em `model/executor.py` e repetir toda a suíte de testes.
2. Validar explicitamente o domínio de `requested_vl`, `sew`, `lmul`, índices de registradores e tamanhos de máscaras.
3. Definir se as instruções de memória terão EEW fixo no mnemônico, como `vle8.v`, `vle16.v` e `vle32.v`, ou se haverá uma operação genérica dependente do SEW ativo. A decisão deve evitar que o nome da instrução contradiga seu comportamento.
4. Generalizar o gerador strip-mined para receber SEW e LMUL e calcular VLMAX, tamanho do elemento e alinhamento de registradores de forma dinâmica.
5. Revisar a representação de `mem_writes` no trace para que seja consistente quanto ao tipo das chaves de endereço e ao formato dos valores.
6. Formalizar a semântica de troca de SEW e LMUL entre instruções, em especial como as políticas de cauda se relacionam com a reinterpretação física dos registradores.
7. Especificar estados de erro e acessos inválidos à memória. O modelo atual usa exceções Python, mas ainda não define equivalentes arquiteturais de traps.
8. Distinguir claramente EEW de memória e SEW. A simplificação atual pode tratar ambos como iguais, mas isso deve ser registrado como limitação até que a semântica RVV correspondente seja implementada.
9. Determinar se a meta é uma extensão customizada inspirada em RVV ou um subconjunto compatível com uma versão específica da especificação RVV. O formato atual de programa Python não é, por si só, um encoding binário RISC-V.

11 Critérios para o próximo marco

A especificação executável poderá ser considerada estável para esta etapa quando:

- todos os testes passarem a partir da raiz do projeto;
- os testes incluírem execução de programas com SEW e LMUL explicitamente variados;
- o strip-mining funcionar com mais de uma configuração de elemento e grupo;

- o trace JSON representar corretamente as escritas de memória;
- as regras de validação estiverem descritas e testadas;
- um ou mais programas de referência tiverem resultados reproduzíveis e documentados.

12 Migração futura para SystemC

A próxima etapa de modelagem deverá preservar a separação conceitual entre:

- estado arquitetural;
- interpretação e validação de instruções;
- execução funcional;
- registro de efeitos observáveis.

A primeira versão SystemC poderá continuar sem modelar temporização detalhada, mas deverá manter o mesmo comportamento arquitetural e produzir resultados comparáveis com os traces do Python. A modelagem de ciclos, latências e protocolos deve ser adicionada em uma etapa separada, para não misturar divergências funcionais com diferenças temporais.

13 Histórico deste documento

Versão	**Descrição**
0.1	Registro inicial dos parâmetros, da semântica funcional e do estado observado do modelo Python. Inclui pendências identificadas na suíte de testes.